

The Rule-of-Two Plugin: Technical Reference

Generated from useful-claude-add-ons@541b2b70 on 2026-09-22. Verify before trusting.

The low-level guide, for the people who install, configure and extend rule-of-two. A companion overview covers the same ground in plain language. Written against rule-of-two 0.1.3, commit 541b2b70.

Design constraint. "Only one reviewer ran" is its own reported outcome, never folded

into a passing report. Coverage is computed by `scripts/rule_of_two.py`, never left to a

model mid-report, and every coverage assertion in the test suite is made against the

rendered report text rather than against the coverage function alone — a value computed

correctly can still be dropped on the way to the page.

1. Layout

Path	Contents
<code>.claude-plugin/plugin.json</code>	Name, version (0.1.3), description, licence (GPL-2.0-only). No hooks registered.
<code>agents/reviewer-claude.md</code>	The Claude-side reviewer. <code>model: fable</code> — a dispatch alias, not a full model id.
<code>commands/review.md</code>	<code>/rule-of-two:review <path></code> .
<code>commands/config.md</code>	<code>/rule-of-two:config</code> .
<code>scripts/rule_of_two.py</code>	Config resolution, family detection, the Codex subprocess wrapper, coverage computation, and report rendering. Single file, no package.

Path	Contents
scripts/_test/ test_rule_of_two.py	The invariant suite plus a --sabotage mode. One test module.
templates/rubric.md	The one written contract both reviewers work from: how to read the artifact, what to run, which failure shapes to hunt, the verdict shape, section order.

Deliberately no agents/reviewer-codex.md: an agent file can only carry model: sonnet|opus|fable, which would be a Claude wrapper wearing an independence label. The Codex review text comes from the codex CLI, invoked directly by rule_of_two.py.

2. The CLI Subcommands

Subcommand	Key arguments	Does
config	none	Prints the resolved config: the two models, their families, the config source, and codex_found.
record-claude	--model, --model-id, --text-file, --log-file, --out, one of --ran/--failed (required, mutually exclusive)	Records the Claude reviewer's outcome. The caller must state success or failure explicitly — it is never inferred from whether the output file is non-empty.
build-prompt	--artifact, --out	Builds the prompt handed to Codex, from the same templates/rubric.md the Claude agent uses.
codex	--prompt-file, --model, --out	Runs run_codex: stdin redirected from /dev/null, an explicit timeout, -s read-only so Codex cannot write anything, not even a scratch directory.
render	--state, --out	Renders a report from a previously assembled state file.

Subcommand

Key arguments

Does

assemble

--artifact, --claude-result, --
codex-result, --out, --state-out

Combines both results, computes coverage, and writes the finished report plus its state.

3. Coverage

Value	Meaning
TWO_CROSS_FAMILY	Both ran, families resolved, families differ. The only outcome where the report uses the word "independent".
TWO_SAME_FAMILY	Both ran, same family — one perspective wearing two names.
TWO_FAMILY_UNVERIFIED	Both ran, but a reviewer's recorded model id does not corroborate the alias beside it — the contradiction is quoted, and the family is not established.
TWO_FAMILY_UNKNOWN	Both ran, but a family would not resolve at all — "could not tell", not a pass.
ONE_REVIEW	One ran. The banner says plainly this is one review, not two, and why the other did not run.
NO_REVIEW	Neither ran.

`compute_coverage(claude_ran, codex_ran, ...)` (`rule_of_two.py:450`) derives the value; `render_title` and the report banner both derive from coverage the same way, rather than one being hand-written prose that could drift from the other.

4. What Counts as a Review That Ran

- `ran` must be the literal boolean `True` — not a truthy string. `"false"`, `"true"`, `1` and `"yes"` are all rejected as the type they are.
- Success is stated by the caller, never inferred from output being non-empty — record-

claude requires --ran or --failed, mutually exclusive, both required.

- A successful run must carry review text. Codex's review is read from --output-last-message, not stdout, because stdout carries a startup banner naming the model and sandbox that an earlier version mistook for findings.
- A review that ran but answers none of the rubric's three verdicts is not silently downgraded to "did not run" — it is flagged as having failed to answer the question, matched as a whole line near the top rather than a substring (a substring match let "This is unviable." satisfy a check for the word VIABLE).

5. Model Identity and Family Resolution

Two separate tables, matched differently on purpose: dispatch aliases (fable, opus, sonnet, haiku, gpt, codex) match as whole words, exactly; model ids match on a prefix that carries its own separator (o3-, never a bare o3), so a loose prefix cannot resolve sonnetting or opus-not-a-real-model to a family. Anything else resolves to UNKNOWN, which is a real value that survives into every line derived from it — never folded into a pass.

verify_model_id checks the alias that was dispatched against the model id the caller separately recorded as having been requested; a mismatch is a contradiction, not silently corroborated, and still produces a full report with the disagreement named in the banner and the offending id left visible rather than dropped.

6. Invoking Codex

run_codex redirects stdin from /dev/null and applies an explicit timeout, mapping a TimeoutExpired to "did not run" rather than "ran and found nothing". The default timeout is 480 seconds; CALLER_TIMEOUT_CEILING_SECONDS (600) states the ceiling it must stay under, because a single Bash call from Claude Code is killed at 600s and a longer configured value could never fire — the caller would kill the command first and the run would be recorded as a tool failure, not as the script's own honest "did not run".

Codex runs with -s read-only, which cannot write even a temporary directory, so it cannot follow the rubric step that needs one, while the Claude reviewer has a Bash tool and can. This asymmetry is reported directly under the coverage banner (render_method_note) rather than equalised, because equalising it (a writable copy of the tree) would still leave the two methods

unequal and would cost a full tree copy to buy an asymmetry that survives it anyway.

7. Configuration

Standalone: never reads crew's `qa.order`, `author_families`, or provider list. Resolution order, nearest first: `.rule-of-two.json` at the repo root, then `~/.claude/rule-of-two/config.json`, then built-in defaults; only the keys present in a layer override the one beneath it. Valid JSON that is not an object falls back to defaults and says so in `config_source` rather than raising three frames down.

```
{

  "claude": {

    "pin": "fable",

    "fallback": "opus",

    "pin_model_id": "claude-fable-5-1",

    "fallback_model_id": "claude-opus-5"

  },
```

```
"codex": { "pin": "gpt-6-astra", "timeout_seconds": 480 }
```

```
}
```

pin/fallback are dispatch aliases; pin_model_id/fallback_model_id are the full ids a CLI --model flag takes. The fallback is driven by /rule-of-two:review step 3, which retries on dispatch_order_claude[1] only when the pinned model was unavailable — never because a review came back thin, which would quietly swap the model the report names without saying so.

8. Running from a Checkout Versus an Install

`\${CLAUDE\_PLUGIN\_ROOT}` is set only when the plugin is installed. Both command files default it and, when empty, walk up from \$PWD looking for plugin/rule-of-two/scripts/rule_of_two.py or a directory that is the plugin itself, failing loudly with "cannot find rule-of-two" if neither resolves. From a plain checkout, config, build-prompt, codex, record-claude, assemble and render all work; the Claude half does not, because the subagent type rule-of-two:reviewer-claude is registered by the plugin and does not exist outside an install — that failure is recorded with --failed and the report states it as a one-reviewer outcome, the same as any other missing reviewer.

9. The Self-Review Record

The first artifact this plugin reviewed was itself, three times, before shipping. Round one: the two reviewers disagreed on the verdict outright and each found blockers the other missed. Round two — reviewing the fix as hard as the first defect — found that a fix applied to one side of a symmetric pair (the loose-prefix match, and a hardened ran check) had not been applied to its neighbour. Round three found a fix that had made the verdict-matching check *worse* — a bulleted list of options now satisfied a check meant to catch "declined to answer" — and a degraded-input fix installed in the wrong function, one caller over from where it was needed. By round three both reviewers converged on the same verdict, which the README treats as the practical argument for the whole design.

10. What Is Not Established

- **Which model actually served the request.** The recorded model is what was configured and asked for, not what a provider is independently attested to have run.
- **Whether Codex will actually work.** `codex_found` is `shutil.which` and nothing else; authentication and model availability fail at dispatch, not at discovery.
- **The fallback retry path.** It is prose instruction to a model (`/rule-of-two:review` step 3), with no code path and therefore no test — a Fable dispatch could not be made to fail on demand to observe it.

Provenance: read from `plugin/rule-of-two/README.md`, `scripts/rule_of_two.py`, `commands/review.md` and `commands/config.md` at commit 541b2b70 (rule-of-two 0.1.3, confirmed in both `.claude-plugin/plugin.json` and `CHANGELOG.md`). Not verified: a live two-reviewer run in this session; exact line numbers for functions cited by name only (`compute_coverage`, `run_codex`, `verify_model_id`) beyond the one explicitly anchored above.